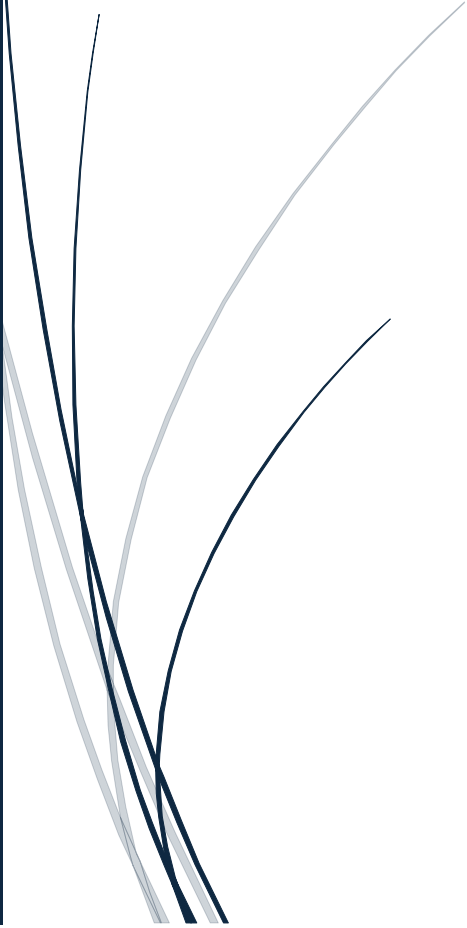




14/09/2026

# Remise 1 - Analyse et planification

Projet SmartDisplay



Cedric Tresor Tchipkap Tchuigoua  
CÉGEP DE RIVIÈRE-DU-LOUP

## Table des matières

|                                                               |   |
|---------------------------------------------------------------|---|
| 1. Compréhension du mandat et du besoin .....                 | 2 |
| 1.1. Contexte .....                                           | 2 |
| 1.2. Besoin .....                                             | 2 |
| 1.3. Entrées du système .....                                 | 2 |
| 1.4. Sorties du système .....                                 | 2 |
| 1.5. Fonctions principales .....                              | 2 |
| 2. Diagramme d'état préliminaire.....                         | 3 |
| 2.1. Explication des états.....                               | 3 |
| 2.2. Conditions de transition principales .....               | 4 |
| 3. Diagramme MQTT et modèles JSON préliminaires .....         | 5 |
| 3.1. Topics MQTT.....                                         | 5 |
| 3.2. Flux de base des échanges de données.....                | 6 |
| 3.3. Modèles JSON préliminaires.....                          | 7 |
| 4. Dépôt Git, répartition des tâches et versionnement.....    | 8 |
| 4.1. Répartition des tâches (projet individuel) .....         | 8 |
| 4.2. Plan de premiers commits (traces de versionnement) ..... | 8 |

## 1. Compréhension du mandat et du besoin

### 1.1. Contexte

Une entreprise souhaite déployer des panneaux d'affichage connectés dans ses locaux afin de diffuser des informations importantes à ses employés. Avant de construire un véritable réseau de panneaux, elle souhaite valider un prototype unique permettant de démontrer la faisabilité technique de la solution.

### 1.2. Besoin

Le prototype doit permettre à un système central d'envoyer un message texte qui sera transmis par Node-RED, publié sur le broker MQTT, reçu par l'ESP32, affiché sur un écran OLED, puis confirmé automatiquement par un accusé de réception. Le système doit aussi distinguer les messages NORMAL des messages ALERTE et permettre la supervision (historique, état du panneau) dans Grafana via InfluxDB.

### 1.3. Entrées du système

- Message texte envoyé depuis Grafana (contenu + type NORMAL/ALERTE)
- Configuration optionnelle (durée d'affichage, luminosité)

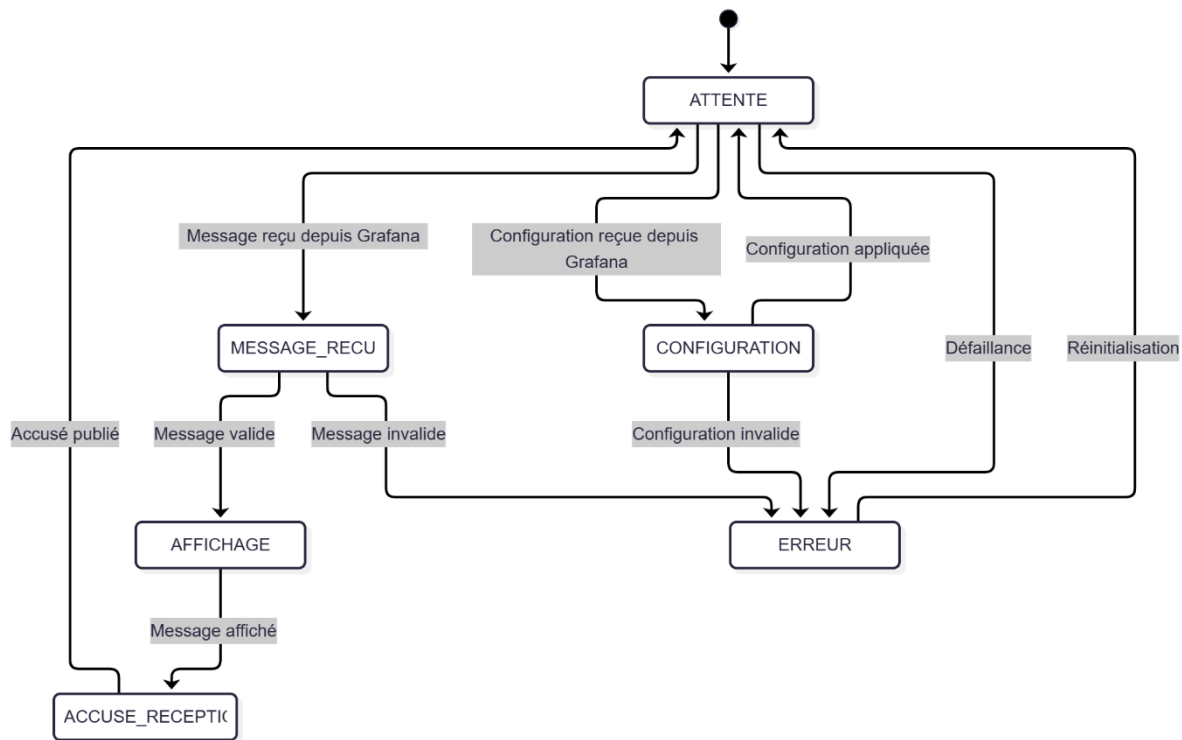
### 1.4. Sorties du système

- Affichage du message sur l'écran OLED
- DEL d'alerte (si message de type ALERTE)
- Publication MQTT de l'accusé de réception
- Publication MQTT de l'état courant du panneau

### 1.5. Fonctions principales

- Se connecter au Wi-Fi et au broker MQTT.
- S'abonner au topic de commande et recevoir les messages.
- Valider et interpréter le message JSON reçu (type, contenu).
- Afficher le message sur l'écran OLED (un seul message à la fois, sans blocage).
- Allumer une DEL si le message est une ALERTE.
- Publier un accusé de réception après affichage.
- Publier l'état du panneau et journaliser dans InfluxDB via Node-RED.
- Gérer les cas d'erreur (message invalide, perte Wi-Fi/MQTT) sans bloquer le système.

## 2. Diagramme d'état préliminaire



### 2.1. Explication des états

| État             | Rôle                                                                                                                                                      |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| ATTENTE          | État de repos ; le panneau écoute les topics MQTT (cmd, config) sans afficher de nouveau message.                                                         |
| MESSAGE_RECUE    | Le message JSON vient d'arriver ; il est validé (champs présents, types corrects).                                                                        |
| AFFICHAGE        | Le message validé est affiché à l'écran OLED (avec DEL activée si type = ALERTE).                                                                         |
| ACCUSE_RECEPTION | Un message MQTT d'accusé de réception est publié pour confirmer l'affichage.                                                                              |
| CONFIGURATION    | Une configuration reçue (durée, luminosité) est appliquée.                                                                                                |
| ERREUR           | État atteint lors d'un message/configuration invalide ou d'une défaillance (Wi-Fi, MQTT). Le système ne bloque pas et peut se réinitialiser vers ATTENTE. |

## 2.2. Conditions de transition principales

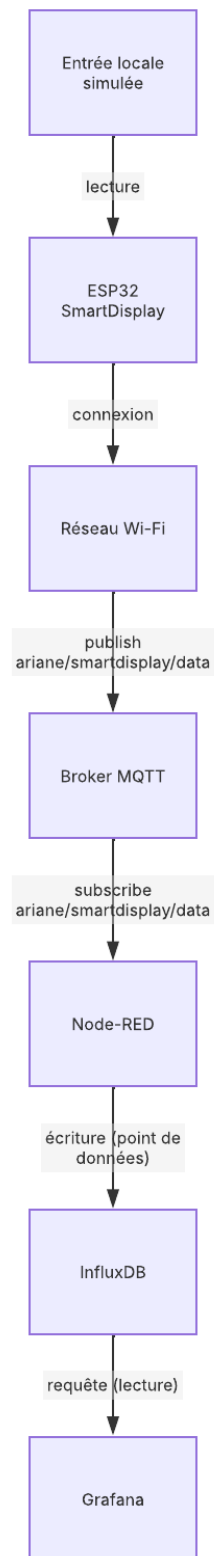
- Message valide : JSON bien formé, champ message non vide, champ type = NORMAL ou ALERTE.
- Message invalide : JSON mal formé, champ manquant ou type inconnu → transition vers ERREUR sans plantage.
- Défaillance : perte de connexion Wi-Fi ou déconnexion du broker MQTT, détectée en continu depuis ATTENTE.
- Réinitialisation : reconnexion réussie ou redémarrage logiciel contrôlé.

### 3. Diagramme MQTT et modèles JSON préliminaires

#### 3.1. Topics MQTT

| Topic                      | Direction                         | Utilisation                                            |
|----------------------------|-----------------------------------|--------------------------------------------------------|
| ariane/smartdisplay/cmd    | Grafana → Node-RED → MQTT → ESP32 | Message à afficher                                     |
| ariane/smartdisplay/config | Grafana → Node-RED → MQTT → ESP32 | Durée d'affichage, luminosité                          |
| ariane/smartdisplay/data   | ESP32 → MQTT → Node-RED           | Message actuellement affiché (journalisation InfluxDB) |
| ariane/smartdisplay/ack    | ESP32 → MQTT → Node-RED           | Accusé de réception après affichage                    |
| ariane/smartdisplay/state  | ESP32 → MQTT → Node-RED           | État courant du panneau (ATTENTE / AFFICHAGE / ERREUR) |

### 3.2. Flux de base des échanges de données



### 3.3. Modèles JSON préliminaires

#### Commande - ariane/smartdisplay/cmd (champs obligatoires : message, type)

```
{  
  "message": "Réunion à 14h00 en salle B",  
  "type": "NORMAL"  
}
```

#### Configuration — ariane/smartdisplay/config (champs optionnels)

```
{  
  "duree_affichage_s": 10,  
  "luminosite_pct": 80  
}
```

#### Accusé de réception - ariane/smartdisplay/ack

```
{  
  "statut": "affiche",  
  "message": "Réunion à 14h00 en salle B",  
  "type": "NORMAL"  
}
```

#### État du panneau - ariane/smartdisplay/state

```
{  
  "etat": "ATTENTE",  
  "dernier_message": "Réunion à 14h00 en salle B",  
  "type": "NORMAL"  
}
```

## 4. Dépôt Git, répartition des tâches et versionnement

### 4.1. Répartition des tâches (projet individuel)

| Tâches                         | Personne qui révisé | Preuve attendue       |
|--------------------------------|---------------------|-----------------------|
| Diagramme de classes           | Auto-révision       | Document et commit    |
| Machine à états                | Auto-révision       | Document et commit    |
| Acquisition des données (MQTT) | Auto-révision       | Code et test          |
| Logique applicative            | Auto-révision       | Code et test          |
| Affichage OLED                 | Auto-révision       | Code et démonstration |
| MQTT et JSON                   | Auto-révision       | Code et messages      |
| Node-RED et InfluxDB           | Auto-révision       | Export et données     |
| Grafana                        | Auto-révision       | Tableau de bord       |
| Tests de robustesse            | Auto-révision       | Fiche complétée       |
| Documentation                  | Auto-révision       | README et rapport     |

Étant seule sur ce mandat, je demeure responsable de chaque tâche ; la révision est réalisée par auto-évaluation (relecture différée de mon propre code, comme prévu à l'Annexe F pour les projets individuels).

### 4.2. Plan de premiers commits (traces de versionnement)

```
git init
git add README.md
git commit -m "chore: initialisation du dépôt et documents d'analyse"

git switch -c feature/init-projet
git add docs
git commit -m "chore: initialisation du dépôt et documents d'analyse (Remise 1)"
git push -u origin feature/init-projet
```